

Learn Low-Level Design in 25 Days: Day 11—Case Study: Designing an Elevator System

Follow the full 25-day series here

TechPreneur

Published October 23, 2025

Free: No

Read on Freedium · original

Contents

The Story: What Happens When You Press the Elevator Button? 0

Step 1: Understanding the Problem Like an Engineer 0

Functional Requirements: 0

Non-Functional Requirements: 0

Step 2: Identify Core Objects 0

1. Elevator 0

2. Request 0

3. ElevatorController 0

4. Floor 0

Step 3: Visualize It 0

Step 4: The Real Challenge — Scheduling 0

Step 5: Handling States 0

Step 6: Bringing It All Together 0

Try It Yourself 0

Key Takeaways 0

What's Next? 0

The Story: What Happens When You Press the Elevator Button?

You press the elevator button. A light turns on. Somewhere, deep inside the building's control system, a decision is made: which elevator should come? What if two are free? What if one is already on its way up? And how does the system ensure it doesn't open doors between floors?

That simple button press kicks off a cascade of design decisions that engineers have to handle — scheduling, direction management, state transitions, and safety logic — all in real time.

Welcome to **Day 1 of "Learn Low-Level Design in 25 Days"** — your hands-on bootcamp to think like a systems designer. Today, we'll start with a **classic case study: designing an elevator system** — one of the most common interview questions and an excellent way to learn object-oriented design, real-world modeling, and how to think in components.



By the end of this post, you'll understand:

- How to identify key objects and relationships in a real-world system

- How to apply OOP principles like encapsulation, abstraction, and responsibility segregation
- How to structure your design to handle scale and complexity

Step 1: Understanding the Problem Like an Engineer

Before jumping into classes and methods, let's reason about **requirements** — not the "what," but the "how."

Functional Requirements:

1. The building has multiple elevators (say 3).
2. Users can request an elevator from any floor (by pressing up/down buttons).
3. Each elevator can move between floors, open/close doors, and serve requests.
4. The system decides **which elevator** should respond to a request.
5. The system should handle concurrent requests efficiently.

Non-Functional Requirements:

- **Responsiveness:** Minimal waiting time.
- **Scalability:** Should work for 3 or 30 elevators.
- **Fault tolerance:** Elevator can go offline without crashing the system.

Let's turn these into a model.

Step 2: Identify Core Objects

Whenever you design a system, ask: "What are the nouns?"

From our story:

- **Elevator**
- **Floor**
- **Request**
- **Controller**

Let's break them down.

1. Elevator

Handles its internal state — moving up/down, opening doors, maintaining direction.

```
class Elevator {
    int currentFloor;
    Direction direction;
    ElevatorState state;
    Queue<Integer> requests;

    void move();
    void openDoor();
    void closeDoor();
    void addRequest(int floor);
}
```

2. Request

Represents a user's call (from outside) or a destination (inside elevator).

```
class Request {
    int floor;
    Direction direction; // only for external requests
}
```

3. ElevatorController

Decides which elevator should handle which request — like the "brain" of the system.

```
class ElevatorController {
    List<Elevator> elevators;

    void requestElevator(int floor, Direction direction);
    Elevator findBestElevator(int floor, Direction direction);
}
```

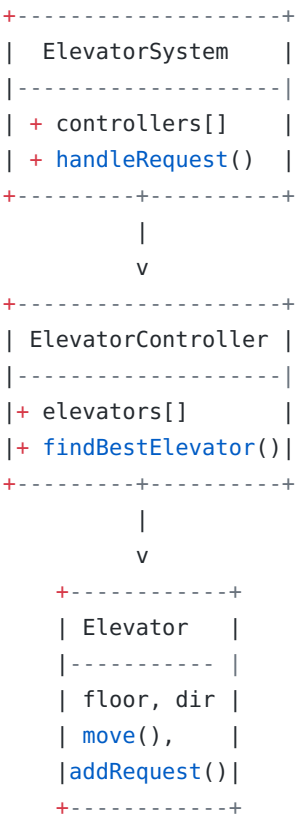
4. Floor

Each floor has buttons for up/down requests.

```
class Floor {
    int floorNumber;
    void pressUp();
    void pressDown();
}
```

Step 3: Visualize It

Here's a simple ASCII snapshot of how these pieces interact:



When a user presses "Up" on Floor 5:

1. The `Floor` sends a request to `ElevatorController`.
2. The controller picks the best `Elevator`.
3. The `Elevator` updates its queue and starts moving.

Step 4: The Real Challenge — Scheduling

This is where good design separates from naive design.

How do we decide **which elevator** takes the request?

Simple heuristics:

- If an elevator is idle → assign immediately.
- If an elevator is moving **toward** the request and will pass that floor → assign it.
- Else → queue it for the nearest idle elevator.

This logic can live inside a **Strategy Pattern**, so we can plug different scheduling algorithms (e.g., SCAN, LOOK, or AI-based in modern systems).

```
interface ElevatorScheduler {
    Elevator selectElevator(List<Elevator> elevators, Request request);
}

class NearestElevatorStrategy implements ElevatorScheduler {
    public Elevator selectElevator(List<Elevator> elevators, Request request) {
        // pick elevator with minimal distance and compatible direction
    }
}
```

Step 5: Handling States

Each elevator can be in states like:

- **Idle**
- **Moving Up**
- **Moving Down**
- **Door Open**
- **Under Maintenance**

```
enum ElevatorState {
    IDLE, MOVING, STOPPED, DOOR_OPEN, MAINTENANCE
}
```

This helps us later extend the logic easily — e.g., if one elevator goes into maintenance, others take over its load.

Step 6: Bringing It All Together

Here's a simplified class interaction flow:

```
User presses UP at Floor 5
     ↓
Floor creates Request(5, UP)
     ↓
ElevatorController.findBestElevator()
     ↓
Elevator.addRequest(5)
     ↓
Elevator.move() → reaches floor → openDoor()
```

This kind of modeling forces you to think in terms of **responsibilities**, not just classes — each object knows what it should know, and nothing more.

Try It Yourself

1. Add an **internal elevator panel** that lets passengers choose floors (1–10).
2. Modify `ElevatorController` to handle **multiple requests concurrently**.
3. How would you design a **maintenance mode** so an elevator can go offline safely?

Key Takeaways

- Always start by identifying **responsibilities**, not classes.
- Separate **control logic** (Controller) from **stateful entities** (Elevator).
- Encapsulate decisions — like scheduling — using **strategy patterns**.
- Keep states explicit — they're critical for debugging and maintenance.
- Good design isn't about "right answers," it's about clear **boundaries and behaviors**.

What's Next?

Tomorrow, in **Day 2: Designing Error Handling and Exception Flows**, we'll explore how to make your systems robust — because every good design should fail gracefully.

Follow the full 25-day series here: Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)